

Relatório de Auditoria de Segurança

Transshipping Desk

29 de agosto de 2026

8	1	4	12
ACHADOS	ALTA	MÉDIA	PONTOS FORTES

Escopo auditado

Auditoria de código do repositório transshippingdesk (branch claude/security-audit-five-categories-esz0rs), cobrindo cinco categorias de falha adaptadas à stack detectada. Foram examinados: as 360 migrations SQL (RLS, policies, grants e funções), as 23 Edge Functions Deno com seus helpers compartilhados, o código-fonte do frontend React, os arquivos de configuração e de deploy, o workflow de CI e os 200 commits do histórico git. A auditoria é estática — nenhum teste foi executado contra ambiente de produção e nenhuma exploração foi realizada.

Stack detectada

Linguagem	TypeScript 6 (strict) — 861 arquivos .ts/.tsx
Frontend	React 19 + Vite 8 + React Router 7 + TanStack Query 5 + Tailwind 4
Backend	Supabase — sem servidor de aplicação próprio
Banco / ORM	PostgreSQL 17 via PostgREST; sem ORM. 360 migrations SQL versionadas
Camada de API	23 Edge Functions Deno + 363 funções PL/pgSQL expostas como RPC
Autenticação	Supabase Auth (JWT). Duas sessões no mesmo projeto: usuário interno e cliente do Portal
Autorização	RLS + GRANT/REVOKE + guardas dentro de funções SECURITY DEFINER
Deploy	Vercel (produção) + Firebase Hosting (rollback) + GitHub Actions (CI)

Nota metodológica

Cada uma das cinco categorias do pedido foi traduzida para o equivalente desta stack antes da varredura. O mapeamento e os limites da cobertura estão registrados abaixo para que o leitor possa julgar o alcance do resultado — inclusive das categorias que não produziram achado.

Como o isolamento foi mapeado

O projeto não tem `tenant_id` nem middleware de tenant: é um sistema operacional interno com um Portal do Cliente acoplado. O isolamento tem duas fronteiras distintas. A primeira é **interno × cliente**: os dois recebem o MESMO role `authenticated` do Supabase Auth, então o role não separa nada — quem separa é o perfil, via `is_active_read_user()` (exige linha ativa em `user_profiles`) e `current_portal_customer_id()` (resolve a conta em `customer_portal_accounts`). A segunda é **cliente × cliente**, aplicada por RLS e pelo escopo dentro de cada RPC do Portal. A auditoria tratou como falha de isolamento todo objeto que confia apenas em "estar autenticado".

Cobertura da varredura

As 360 migrations foram reproduzidas em ordem por um script de replay que resolve o estado **final** de cada policy e de cada GRANT — inclusive os criados dentro de blocos `DO $$ ... FOREACH ... $$`, que uma leitura por `grep` não enxerga. Sobre esse estado final foram cruzadas 92 tabelas × RLS, 282 policies vivas e 363 funções × grants × guardas, com resolução recursiva de cadeias de delegação (uma função sem guarda própria que delega a uma função guardada não é achado). As 23 Edge Functions e os 1.717 linhas de seus helpers foram lidas integralmente, não amostradas.

Critério de achado

Só entrou no relatório o que foi confirmado no código com arquivo e linha. Hipóteses não confirmadas foram descartadas em vez de reportadas com ressalva. Onde a categoria não produziu achado, isso é dito explicitamente e acompanhado da evidência que sustenta a conclusão — a ausência de achado é um resultado da auditoria, não uma lacuna dela.

Mapeamento das categorias

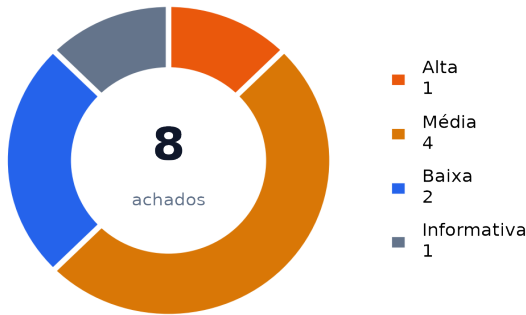
Categoria	Equivalente auditado nesta stack
1. Banco sem tranca (isolamento de inquilino/dono)	Em Supabase, o mecanismo é RLS + guardas em funções SECURITY DEFINER. Uma função SECURITY DEFINER ignora RLS por definição: se ela for concedida a authenticated e não tiver guarda própria, é leitura livre para o cliente do Portal.
2. Permissão definida no navegador	Cruzamento de cada gate de papel do frontend (roleHasPermission, ProtectedRoute adminOnly, can()) com o endpoint correspondente (Edge Function ou RPC).
3. IDOR	Toda RPC e toda Edge Function que recebe um identificador de objeto por path, query ou body, verificando se o objeto é confrontado com o dono/tenant do chamador.
4. Chaves expostas (hardcode)	Código-fonte, configs, supabase/config.toml, vercel.json, firebase.json, CI, scripts, documentação, defaults de variável de ambiente e os 200 commits do histórico git.
5. Inputs sem tratamento (XSS)	Frontend: dangerouslySetInnerHTML, innerHTML, eval, href/src controlados por usuário. Backend: input de usuário em HTML de e-mail e em respostas.

Resumo executivo

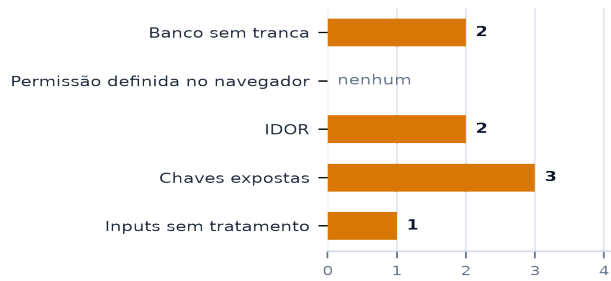
A auditoria registrou **8 achados**: **nenhum crítico**, **1 de severidade alta**, **4 médios**, **2 baixos** e **1 informativo**. Também foram registrados **12 pontos fortes verificados**, que sustentam a cobertura da varredura e explicam por que uma base deste tamanho — 92 tabelas, 363 funções de banco e 23 Edge Functions — produziu tão poucos achados.

O projeto tem histórico visível de endurecimento: várias migrations existem apenas para fechar brechas encontradas em auditorias anteriores, e a maior parte delas documenta no cabeçalho o que foi fechado e por quê. Os achados aqui não contradizem esse trabalho — **seis dos oito são resíduos dele**. São funções que uma correção anterior identificou como problema, corrigiu para o caminho não autenticado e deixou abertas para o papel authenticated, ou que ficaram de fora de uma revogação em lote porque o nome não casava com o padrão usado. O risco central não é uma falha de projeto: é a ausência de verificação automatizada de um invariante que hoje depende de disciplina humana repetida em 201 funções.

Achados por severidade



Achados por categoria



Índice dos achados

A coluna **Cat.** remete à categoria do pedido, detalhada na nota metodológica. Cada achado é desenvolvido, com trecho de código e correção, na seção “Achados detalhados”.

#	Severidade	Achado	Cat.
1	ALTA	portal_billing_gate expõe e-mail e identificadores de qualquer cliente por B/L arbitrário	3
2	MÉDIA	refresh_voyage_status_from_terminal_scales altera o status de qualquer viagem sem verificação de privilégio	3
3	MÉDIA	Leitura de programação de viagem sem guarda contorna o portão show_on_portal	1
4	MÉDIA	portal_check_auth_method: oráculo de enumeração de clientes e vazamento de portal_email (código morto ainda publicado)	1
5	MÉDIA	Segredo de webhook e de cron caem para a chave service_role quando não configurados	4
6	BAIXA	portal-daily-digest compara o segredo com != em vez de comparação em tempo constante	4
7	BAIXA	Segredos fixos em scripts do ambiente local de auditoria de design	4
8	INFORMATIVA	innerHTML reinjetado em document.write no fluxo de impressão (sink latente, não explorável hoje)	5

Pontos fortes verificados

Registrado com evidência de arquivo e linha. Esta seção é a prova de cobertura da auditoria: cada item abaixo é um controle que foi procurado, encontrado e confirmado funcionando — não uma ausência de achado.

RLS universal e sem policy permissiva

Replay das 360 migrations: **92 de 92 tabelas** com `ENABLE ROW LEVEL SECURITY` e **282 policies vivas**. Nenhuma sobrevivente com `USING (true)` ou `auth.role() = 'authenticated'`. As policies permissivas originais de `002_rls.sql` foram substituídas por `010_rls_by_role.sql`, e as que a migration 042 deixou escapar por erro de nome (`vazios_imp_*`) foram removidas pela 097.

Tabelas sensíveis falham fechadas

Seis tabelas têm RLS habilitada e **zero policies** — logo, negam tudo para quem não é `service_role`: `portal_login_attempts`, `portal_login_resolution_attempts`, `portal_rate_limits`, `portal_notifications`, `provision_rate_limit_log` e `agency_report_pending_baselines`. É o padrão correto para tabela de contador de rate limit.

Gestão de usuários verifica privilégio no servidor

`supabase/functions/admin-users/index.ts:36-41` chama `is_admin()` com o JWT do chamador e devolve 403 antes de qualquer operação; só então instancia o cliente `service_role`. A desativação de usuário (`:134-138`) é feita **pelo cliente do chamador**, de propósito, para que a policy de admin continue valendo e o trigger de auditoria enxergue `auth.uid()`. O gate `adminOnly` do frontend tem, portanto, equivalente real no servidor.

As quatro permissões do frontend têm contraparte no banco

`roleHasPermission` (`src/hooks/useAuth.tsx:19-34`) define `admin_panel`, `manage_users`, `portal_provisioning` e `settle_financial_adjustments`. Todas verificadas: as duas primeiras por `is_admin()` na Edge Function; a terceira por `portal_current_role` em `portal-invite-send:19-20` e `portal-account-suspend:11-12`; a quarta por `is_financeiro_user()` em `313_cod_adjustment_settlement.sql:351-353`.

IDOR do Portal fechado por construção, não por revisão caso a caso

As RPCs de dados do Portal não recebem `customer_id`: elas resolvem o cliente por `current_portal_customer_id()` e repassam a um núcleo `_portal_*_core`. As funções de inspeção interna recebem o `customer_id`, mas só através de `_portal_inspect_guard()` (`292_portal_inspection.sql:27-37`), que exige `is_active_read_user()`. Os núcleos `_core`, que aceitam `customer_id` arbitrário, são revogados de `authenticated` em bloco (`292:207-214`). O parâmetro perigoso é inalcançável.

Escrita do Portal valida posse item a item

`portal_create_consolidation` recusa a operação inteira com 42501 se qualquer recebível da seleção pertencer a outro cliente (`br.customer_id <> v_customer_id`), e ainda exige CE Mercante liberado por B/L. `portal_mark_notification_read`, `portal_obsolete_consolidation` e `portal_request_dispute_reopen` filtram pelo cliente resolvido.

HTML de e-mail totalmente escapado

`supabase/functions/_shared/portalEmailTemplates.ts:3` define `escapeHtml` e o aplica em **todos** os pontos de interpolação do HTML: parágrafos, nome da empresa, CNPJ, rótulos, valores, itens de lista, rótulo do botão e — o que costuma faltar — a **URL do botão** (`:57`). Nome de cliente vem de importação de planilha, então é input não confiável, e está coberto.

Nenhum sink clássico de XSS no frontend

Varredura de `src/`: zero ocorrências de `dangerouslySetInnerHTML`, `eval`, `new Function` ou `insertAdjacentHTML`. Não há renderização de Markdown. Os únicos `href` dinâmicos (`ChegadasSaidas.tsx:322`, `ShipScheduleWidget.tsx:67`) montam URL do `MarineTraffic` a partir de um número IMO, sem espaço para `javascript`:

Histórico git limpo e `.env` protegido

Nenhum JWT ou chave em 200 commits (busca por padrão `eyJ...eyJ...` em toda a árvore de revisões). O único arquivo de ambiente versionado é `.env.example`, com placeholders; `.gitignore:16-18` cobre `.env` e `.env.*`. O CI injeta as chaves por `secrets.*`, e apenas as duas variáveis `VITE_` públicas por natureza.

Cabeçalhos de segurança e CSP restritiva

`vercel.json` e `firebase.json` aplicam `script-src 'self'` (sem `unsafe-inline` e sem `unsafe-eval`), `object-src 'none'`, `frame-ancestors 'none'`, `base-uri 'self'`, além de `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff` e `Permissions-Policy`. O `connect-src` lista os destinos exatos.

Autenticação do Portal endurecida contra enumeração e força bruta

O login devolve mensagem genérica única, contabiliza tentativas por CNPJ (`portalLoginRateLimit.ts`, com falha da RPC contando como bloqueio) e iguala o **tempo** de resposta entre os desfechos via `EdgeRuntime.waitUntil` — fechando o oráculo por tempo que uma auditoria anterior encontrou. Tokens de convite e recuperação são aleatórios de 32 bytes, armazenados apenas como SHA-256, de uso único e com consumo condicional (`UPDATE ... WHERE status = 'pendente'`), o que resolve a corrida.

CORS por allowlist, negando pela ausência do cabeçalho

`_shared/cors.ts:53-61` só emite `Access-Control-Allow-Origin` quando a origem está na allowlist — em vez de devolver a string `'null'`, que casaria com iframes sandbox e documentos `data:`. O padrão de preview do Vercel é ancorado ao projeto e à equipe, não a um wildcard de `vercel.app`, e `Vary: Origin` acompanha.

Pontos fracos — os riscos centrais

Os achados individuais estão detalhados na seção seguinte. Aqui ficam os três padrões estruturais dos quais eles derivam — corrigir a estrutura vale mais do que corrigir cada ocorrência.

1. O role `authenticated` é compartilhado entre operador e cliente

É a fragilidade estrutural da qual derivam quatro dos oito achados. O cliente do Portal e o operador interno recebem o mesmo role do Supabase Auth; a separação real é feita **função a função**, por guarda no corpo ou por ACL. Com **201 funções SECURITY DEFINER concedidas a `authenticated`**, o custo de errar é uma função nova sem guarda — e cada uma delas ignora a RLS por definição. A disciplina hoje é humana e não tem verificação automatizada.

2. Código morto continua publicado no PostgREST

O fluxo de token legado do Portal (cinco funções) e `portal_billing_gate` não têm chamador de produção, mas seguem com EXECUTE para `authenticated` e, portanto, publicados como endpoint HTTP. As migrations 093 e 296 já identificaram esse resíduo por escrito — a 296 chega a chamá-lo de "o resíduo que vira vulnerabilidade quando alguém reaproveita a função legada mais tarde" — mas fecharam apenas os nomes que casavam com um padrão, e estes ficaram de fora.

3. Degradação silenciosa de configuração de segredo

Duas Edge Functions aceitam a `service_role` como bearer quando o segredo dedicado não está definido, sem nenhuma validação de inicialização que recuse esse estado. A ausência de configuração não é distinguível da configuração correta em tempo de execução.

Achados detalhados

Agrupados pela categoria do pedido. Cada achado traz arquivo e linha exatos, o trecho de código, por que é explorável, o impacto, as condições de exploração e a correção sugerida.

1. Banco sem tranca (isolamento de inquilino/dono)

MÉDIA

Achado 3 — Leitura de programação de viagem sem guarda contorna o portão show_on_portal

Local: supabase/migrations/326_voyage_operation_alerts.sql:15-49 e :53-103

```
-- 1. Helper: PODs elegíveis da viagem
CREATE OR REPLACE FUNCTION public.get_voyage_eligible_pods(p_voyage_id BIGINT)
RETURNS TABLE(pod TEXT)
LANGUAGE sql STABLE SECURITY DEFINER
SET search_path = public, pg_temp
AS $$
    WITH latest_states AS (
        SELECT DISTINCT ON (entity_id, field_name) entity_id, field_name, new_value
        FROM public.audit_logs
        WHERE entity_type = 'voyage_pod_schedule'
        AND entity_id LIKE p_voyage_id || ':%'
        ORDER BY entity_id, field_name, new_value DESC
    )
    ...
    $$;
REVOKE ALL ON FUNCTION public.get_voyage_eligible_pods(BIGINT) FROM PUBLIC, anon;
GRANT EXECUTE ON FUNCTION public.get_voyage_eligible_pods(BIGINT) TO authenticated, service_role
;

-- 2. Helper: menor ETA brasileiro
CREATE OR REPLACE FUNCTION public.get_voyage_first_brazilian_eta(p_voyage_id BIGINT)
RETURNS DATE ... SECURITY DEFINER ... -- idem: sem guarda
GRANT EXECUTE ON FUNCTION public.get_voyage_first_brazilian_eta(BIGINT) TO authenticated, service_role;
```

Por que é explorável

As duas funções são SECURITY DEFINER sem guarda, concedidas a authenticated, e leem audit_logs — cuja RLS restringe a leitura a usuário interno ativo. Por serem DEFINER, essa RLS não se aplica.

O ponto que torna isso uma falha de isolamento, e não apenas uma inconsistência, é o portão voyages.show_on_portal (migration 172), criado exatamente para decidir qual programação o cliente pode ver, e reforçado pela migration 257, que removeu as policies USING (true) de vessel_schedules por contornarem esse portão. Estas duas funções contornam o mesmo portão pelo outro lado: um cliente do Portal itera voyage_id (inteiro sequencial) e obtém os portos de escala elegíveis e a data de ETA de qualquer viagem, inclusive das que a operação decidiu não publicar.

Impacto

Vazamento de programação comercial (rota e datas) que o portão de publicação existe para controlar. Para um cliente que é concorrente de outro embarcador na mesma rota, é informação de valor competitivo.

Condições de exploração

Qualquer sessão autenticada. O retorno é limitado a códigos de porto e datas — não expõe B/L, cliente nem valores.

Correção sugerida

São helpers de detector: o consumidor natural é `service_role`. Revogue `authenticated` das duas. Se a UI interna precisar delas, adicione `WHERE public.is_active_read_user()` no corpo SQL — o mesmo padrão que a migration 257 aplicou em `list_billing_runs`.

MÉDIA

Achado 4 — `portal_check_auth_method`: oráculo de enumeração de clientes e vazamento de `portal_email` (código morto ainda publicado)

Local: `supabase/migrations/053_security_hardening.sql:16-49`

```
CREATE OR REPLACE FUNCTION public.portal_check_auth_method(p_cnpj_cpf TEXT)
RETURNS JSONB LANGUAGE plpgsql SECURITY DEFINER
SET search_path = public, pg_temp
AS $$
BEGIN
    SELECT a.auth_user_id, a.portal_email, a.active INTO v_account
    FROM public.customer_portal_accounts AS a
    JOIN public.customers AS c ON c.id = a.customer_id
    WHERE c.cnpj_cpf = p_cnpj_cpf;          -- <== CNPJ arbitrário, dado público

    IF NOT FOUND OR NOT v_account.active THEN
        RETURN jsonb_build_object('method', 'none');
    END IF;
    IF v_account.auth_user_id IS NOT NULL THEN
        RETURN jsonb_build_object('method', 'supabase_auth',
                                   'portal_email', v_account.portal_email);
    END IF;
    RETURN jsonb_build_object('method', 'legacy_token');
END;
$$;

REVOKE ALL ON FUNCTION public.portal_check_auth_method(TEXT) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.portal_check_auth_method(TEXT) TO anon, authenticated;
```

Por que é explorável

A própria migration 053 nasceu para fechar a enumeração aqui: o cabeçalho diz "remove user enumeration (...) impedindo varredura de CNPJs de clientes via API anon". A correção unificou a resposta de CNPJ inexistente e conta inativa, e a migration 093 depois revogou `anon` de todas as funções `SECURITY DEFINER`.

O `grant a authenticated` **nunca foi revogado**, e a unificação não cobre o terceiro desfecho: conta **ativa** devolve `method: supabase_auth` mais o `portal_email`.

O CNPJ é dado público (consultável na Receita Federal), então o parâmetro não é segredo. Qualquer sessão autenticada — inclusive um cliente do Portal — varre uma lista de CNPJs e aprende, para cada um, se a empresa tem conta ativa no Portal e qual o endereço registrado. A função é além disso **código morto**: a própria migration 093 registra que ela é "do fluxo de token legado e não tem mais nenhum call site no frontend", o que a auditoria confirmou — o frontend usa `portal_get_session_overview_v2`.

Impacto

Enumeração da carteira de clientes com Portal ativo e coleta dos endereços associados. É o mesmo risco que a migration 053 tentou eliminar, sobrevivendo por um `grant` que ela não tocou. Superfície de ataque mantida por código sem consumidor.

Condições de exploração

Qualquer sessão autenticada. A migration 093 já fechou o caminho não autenticado (`anon`), então o pré-requisito é ter uma conta — o que um cliente do Portal tem por definição.

Correção sugerida

Revogar e, de preferência, remover. O fluxo de token legado inteiro (`portal_check_auth_method`, `portal_get_session_overview`, `portal_list_pending_bls`, `portal_logout`, `resolve_customer_portal_session`) está sem chamador; a migration 296 já aplicou essa disciplina às funções `*_legacy` e estas ficaram de fora só porque o nome não casa com o padrão.

2. Permissão definida no navegador

Nenhum achado. A categoria se aplica à stack e foi verificada exaustivamente: o modelo de permissão do frontend é pequeno e fechado — quatro permissões em `roleHasPermission` (`src/hooks/useAuth.tsx:14-34`) e uma única rota `adminOnly` (`src/App.tsx:202-206`, `/admin/usuarios`). Cada uma das quatro foi cruzada com o endpoint que executa a operação e **todas têm verificação equivalente no servidor**, detalhada nos pontos fortes. Vale registrar que `ProtectedRoute` é honesto quanto ao seu papel: ele redireciona a UI, e a autorização real está no banco — que é exatamente a postura correta para uma SPA estática.

3. IDOR

ALTA

Achado 1 — `portal_billing_gate` expõe e-mail e identificadores de qualquer cliente por B/L arbitrário

Local: `supabase/migrations/325_clientes_portal_disputes_alerts.sql:24-63` (definição) e `:67` (grant)

```
CREATE OR REPLACE FUNCTION public.portal_billing_gate(p_bl_id TEXT)
RETURNS JSONB
LANGUAGE plpgsql STABLE SECURITY DEFINER
SET search_path = public, pg_temp
AS $function$
...
  SELECT customer_id INTO v_customer_id FROM public.bls WHERE id = p_bl_id;
...
  RETURN jsonb_build_object(
    'allowed', v_reason IS NULL,
    'reason', v_reason,
    'customer_id', v_customer_id,
    'account_id', v_account.id,
    'recovery_email', v_email          -- <= e-mail real do cliente
  );
END;
$function$;

REVOKE ALL ON FUNCTION public.portal_billing_gate(TEXT) FROM PUBLIC, anon;
GRANT EXECUTE ON FUNCTION public.portal_billing_gate(TEXT) TO authenticated, service_role;
```

Por que é explorável

A função é **SECURITY DEFINER** — portanto ignora a RLS de `bls` e de `customer_portal_accounts` — está concedida a `authenticated` e **não tem nenhuma guarda no corpo**: não chama `is_active_read_user()`, não chama `current_portal_customer_id()`, e não confronta o `customer_id` do B/L com o do chamador. O único parâmetro é o ID do B/L, escolhido livremente por quem chama.

O cliente do Portal recebe exatamente o mesmo `role authenticated` que o operador interno (ADR 0013 e migration 257), e o PostgREST publica toda função concedida a esse `role`. Um cliente autenticado no Portal

chama, direto por HTTP, POST /rest/v1/rpc/portal_billing_gate com o ID de um B/L que não é dele e recebe de volta o customer_id, o account_id e o **e-mail de recuperação** do cliente dono daquele B/L. O ID do B/L é textual e sequencial no formato usado pelo próprio projeto (CSC0000000001, CSC0000000002 em .env.example), o que torna a enumeração trivial: um laço sobre a faixa devolve o cadastro de contato de toda a carteira.

Impacto

Vazamento cross-tenant de PII (e-mail de contato) e do mapa cliente↔B/L de toda a operação. O e-mail de recuperação é justamente o endereço que recebe os links de recuperação de senha do Portal, o que transforma o vazamento em insumo direto para phishing dirigido contra a conta correspondente.

Condições de exploração

Nenhuma condição especial. Basta uma conta de Portal ativa (ou qualquer sessão interna). Não depende de feature flag nem de configuração insegura.

Correção sugerida

A função só tem consumidor interno (o gate de faturamento). Revogue o acesso do papel compartilhado e mantenha apenas service_role, como as migrations 257 e 296 já fizeram com bl_has_portal_release e check_provision_rate_limit:

```
REVOKE ALL ON FUNCTION public.portal_billing_gate(TEXT) FROM PUBLIC, anon, authenticated;
```

Se algum chamador interno precisar dela com sessão de usuário, adicione no início do corpo a guarda padrão do projeto: IF NOT public.is_active_read_user() THEN RAISE EXCEPTION 'Sem permissão.' USING ERRCODE = '42501'; END IF;

MÉDIA

Achado 2 — refresh_voyage_status_from_terminal_scales altera o status de qualquer viagem sem verificação de privilégio

Local: supabase/migrations/342_atracacao_alertas.sql:7-44 (definição) e :47 (grant)

```
CREATE OR REPLACE FUNCTION public.refresh_voyage_status_from_terminal_scales(p_voyage_id BIGINT)
RETURNS VOID
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, pg_temp
AS $function$
...
BEGIN
  -- nenhuma verificação de identidade ou de papel
  ...
  IF v_active_scales > 0 THEN
    UPDATE public.voyages
    SET status = CASE WHEN v_pending_scales = 0 THEN 'completed' ELSE 'active' END
    WHERE id = p_voyage_id AND status <> 'cancelled';
  END IF;
END;
$function$;

REVOKE ALL ON FUNCTION public.refresh_voyage_status_from_terminal_scales(BIGINT) FROM PUBLIC, anon;
GRANT EXECUTE ON FUNCTION public.refresh_voyage_status_from_terminal_scales(BIGINT) TO authenticated, service_role;
```

Por que é explorável

É uma função de **escrita**, SECURITY DEFINER (portanto acima da RLS de viagens), concedida a `authenticated`, que recebe o `voyage_id` como único parâmetro e não verifica nem quem chama nem se o chamador tem relação com aquela viagem. Ela é o reconciliador chamado por triggers e pelo detector agendado — mas o grant a `authenticated` a torna também um endpoint público do PostgREST.

Qualquer sessão autenticada, inclusive a de um cliente do Portal, pode chamá-la em laço sobre IDs de viagem (inteiros sequenciais) e forçar a transição de status para `completed` ou `active`. Vale notar que a escrita não é arbitrária: o valor gravado é derivado do estado real das escalas, então o atacante não escolhe o status. O que ele obtém é a capacidade de **disparar a transição fora de hora** e de resolver, pelo mesmo caminho, os itens de alerta associados — em cima de qualquer viagem da operação.

Impacto

Integridade do estado operacional e da fila de alertas. Uma viagem marcada como `completed` antes do tempo, ou um alerta operacional resolvido por um terceiro, corrompe o painel que a operação usa para decidir. Como a escrita não passa pelo caminho normal, ela também não deixa o rastro de autoria que os fluxos internos deixam.

Condições de exploração

Nenhuma condição especial; qualquer sessão autenticada. O efeito depende do estado das escalas da viagem alvo, então nem toda chamada produz mudança.

Correção sugerida

Manter o grant apenas para `service_role` (o detector agendado) ou, se houver chamador interno com sessão de usuário, adicionar a guarda que as funções irmãs da mesma família já usam — `resolve_alert_item` valida e levanta 42501:

```
IF auth.uid() IS NULL OR NOT public.is_active_user() THEN RAISE EXCEPTION 'Sem permissão.' USING ERRCODE = '42501'; END IF;
```

4. Chaves expostas (hardcode)

MÉDIA

Achado 5 — Segredo de webhook e de cron caem para a chave `service_role` quando não configurados

Local: `supabase/functions/notify-invoice-issued/index.ts:77-78` e `supabase/functions/recalc-demurrage-ptax/index.ts:61-62`

```
// supabase/functions/notify-invoice-issued/index.ts:77-78
const webhookSecret =
  Deno.env.get('NOTIFY_WEBHOOK_SECRET') ?? Deno.env.get('SUPABASE_SERVICE_ROLE_KEY') ?? ''

// supabase/functions/recalc-demurrage-ptax/index.ts:61-62
const serviceRoleKey = Deno.env.get('SUPABASE_SERVICE_ROLE_KEY') ?? ''
const cronSecret = Deno.env.get('RECALC_CRON_SECRET') ?? serviceRoleKey
```

Por que é explorável

Não é um segredo escrito no código, e sim o padrão que o pedido da auditoria chama de *default que vira segredo real se não for sobrescrito* — com o agravante de que o default aqui é a credencial **mais privilegiada do projeto**. Se `NOTIFY_WEBHOOK_SECRET` ou `RECALC_CRON_SECRET` não estiver definida, a função passa a aceitar como bearer a `SUPABASE_SERVICE_ROLE_KEY`. Para o webhook funcionar nesse estado, um operador tem de **colar a service_role key no campo de secret header** do Database Webhook e do

agendador — e a partir daí ela viaja como bearer em cada disparo, fica gravada na configuração do painel e tende a aparecer em log de requisição.

Uma chave que só deveria existir dentro do runtime da função passa a circular como token de transporte. Não há nenhuma validação de inicialização que recuse essa configuração: o código não distingue "segredo dedicado ausente" de "segredo dedicado configurado", então a degradação é silenciosa. Note que isto **não** é um bypass de autenticação — quem não conhece nenhuma das duas chaves continua recebendo 401.

Impacto

Ampliação da superfície de exposição da credencial que ignora toda a RLS do projeto. O comprometimento dela é total: leitura e escrita irrestritas em todas as 92 tabelas.

Condições de exploração

Requer a configuração insegura (variável dedicada ausente) para se materializar. Não é explorável remotamente por si só; é uma falha de higiene de credencial que amplia o raio de outro incidente.

Correção sugerida

Remover o fallback e falhar de forma observável, como `_shared/portalEmail.ts:47` já faz com `PORTAL_FROM_EMAIL`:

```
const webhookSecret = Deno.env.get('NOTIFY_WEBHOOK_SECRET')
if (!webhookSecret) return json(500, { error: 'Webhook não configurado.' })
Depois, girar a service_role key se ela já tiver sido usada como secret header em produção.
```

BAIXA

Achado 6 — portal-daily-digest compara o segredo com `!=` em vez de comparação em tempo constante

Local: `supabase/functions/portal-daily-digest/index.ts:7-9`

```
const expectedSecret = Deno.env.get('PORTAL_DIGEST_SECRET')
const providedSecret = req.headers.get('Authorization')?.replace(/^Bearer\s+/i, '')
if (!expectedSecret || providedSecret !== expectedSecret) return new Response(JSON.stringify({ e
  rror: 'Unauthorized' })), { status: 401 })
```

Por que é explorável

É a única das quatro funções protegidas por bearer secret que compara com `!==`. As outras três — `alerts-detector:3-11`, `notify-invoice-issued:58-66` e `recalc-demurrage-ptax:26-34` — implementam `timingSafeEqual`, e a de `notify-invoice-issued` até documenta o motivo em comentário. A comparação de strings do JavaScript sai no primeiro byte diferente, o que em tese permite recuperar o segredo byte a byte pela variação do tempo de resposta.

Na prática a exploração é pouco realista: o ruído de rede e o cold start do Edge Runtime dominam a diferença de nanossegundos. O achado é registrado como **inconsistência de padrão**: a defesa foi adotada como convenção no projeto e este ponto ficou de fora.

Impacto

Baixo. Recuperação teórica do `PORTAL_DIGEST_SECRET`, que dispara o envio do resumo diário aos usuários internos.

Condições de exploração

Exige medição de tempo através da rede contra ruído alto. Não demonstrada.

Correção sugerida

Extrair o `timingSafeEqual` já duplicado em três funções para `_shared/` e usá-lo nas quatro — resolve o achado e remove a triplicação no mesmo movimento.

BAIXA

Achado 7 — Segredos fixos em scripts do ambiente local de auditoria de design

Local: `scripts/design-audit/sb-shim.cjs:18` e `scripts/design-audit/win/local-stack.ps1:32,128`

```
// scripts/design-audit/sb-shim.cjs:1-2 e :18
// Minimal local Supabase emulator (PostgREST subset + GoTrue password auth)
// for UI auditing against a local Postgres. Not for production use.
const JWT_SECRET = 'local-audit-jwt-secret-local-audit-jwt-secret-32'

# scripts/design-audit/win/local-stack.ps1:32
$env:PGPASSWORD = "postgres"
```

Por que é explorável

São credenciais fixas de um emulador local do Supabase usado pela skill de auditoria de design. O próprio arquivo se declara "Not for production use" na segunda linha, o shim escuta em `127.0.0.1:54321` e nada em `vercel.json`, em `firebase.json` ou no workflow de CI o executa ou o publica.

O registro é higiene, não vulnerabilidade explorável: um segredo de assinatura de JWT literal no repositório é exatamente o padrão que uma varredura automatizada de segredos aponta, e convém que o motivo de ele ser inofensivo esteja escrito em vez de precisar ser redescoberto a cada auditoria.

Impacto

Nenhum em produção. O risco real é de contágio: alguém reaproveitar o shim ou o script fora do contexto local.

Condições de exploração

Não explorável remotamente. Exige que o shim seja executado e exposto fora de localhost — o que nenhum caminho do projeto faz.

Correção sugerida

Ler as duas credenciais de variável de ambiente com default apenas quando `NODE_ENV !== 'production'`, e recusar a inicialização fora de localhost. Alternativamente, manter como está e registrar a exceção no arquivo de configuração da varredura de segredos, para que o achado não volte a cada execução.

5. Inputs sem tratamento (XSS)

INFORMATIVA

Achado 8 — `innerHTML` reinjetado em `document.write` no fluxo de impressão (sink latente, não explorável hoje)

Local: `src/lib/printDocument.ts:8` e `src/pages/Demurrage.tsx:157`

```
// src/lib/printDocument.ts:1-8
export function printDocumentElement(element: HTMLElement, title: string) {
  const printWindow = window.open('', '_blank', 'width=900,height=1100')
  ...
  printWindow.document.write(`<!doctype html><html><head><title>${title}</title>
  ...<div class="invoice-print-content">${element.innerHTML}</div></body></html>`)

// src/pages/Demurrage.tsx:157 — mesma construção, título literal
printWindow.document.write(`...<title>Fatura de Demurrage</title>...
  <div class="invoice-print-content">${content.innerHTML}</div>...`)
```

Por que é explorável

Registrado como informativo porque **não é explorável no estado atual**, e a auditoria confirmou as três condições que sustentam isso: (a) não existe nenhum `dangerouslySetInnerHTML` no repositório, então o `innerHTML` lido é DOM que o React renderizou e já escapou; (b) o `title` interpolado sem escape em `printDocument.ts:8` não chega a ser um vetor porque a função **não tem nenhum chamador** — é código morto; (c) em `Demurrage.tsx:157`, que é o caminho vivo, o título é uma string literal.

O que se registra é a fragilidade da construção. A janela aberta por `window.open('')` é **same-origin**: se um dia alguém introduzir um `dangerouslySetInnerHTML` na árvore do documento de impressão, ou passar um título vindo do usuário para a função morta, o XSS resultante roda no mesmo contexto da aplicação, com acesso ao token da sessão no storage. O projeto não tem biblioteca de sanitização instalada, então não existe rede de segurança para esse caso.

Impacto

Nenhum hoje. O valor do registro é impedir que a construção se torne o vetor de uma mudança futura aparentemente inócua.

Condições de exploração

Não explorável no código atual. Depende da introdução de um sink de HTML bruto na árvore impressa ou de um chamador que passe título controlado pelo usuário.

Correção sugerida

Remover `printDocumentElement`, que não tem chamador, ou escapar o `title` antes de interpolar. A CSP de `vercel.json` (`script-src 'self'`) já bloqueia script inline e é a mitigação que hoje segura o caso — vale registrar essa dependência no comentário da função.

Recomendações priorizadas

A prioridade combina severidade, custo de exploração e custo de correção. Os itens P1 são alterações de uma linha de SQL cada, sem impacto em consumidor de produção — o melhor retorno da lista.

Prio.	Ação	Ref.	Justificativa
P1	Fechar o ACL de <code>portal_billing_gate</code>	Achado 1	Único achado de severidade alta e o de exploração mais barata: vaza PII cross-tenant com uma chamada HTTP e um ID sequencial. É uma linha de SQL — <code>REVOKE ... FROM authenticated</code> — sem impacto em consumidor de produção.
P1	Guardar <code>refresh_voyage_status_from_terminal_scales</code>	Achado 2	É a única escrita não autorizada da auditoria. Manter apenas <code>service_role</code> ou adicionar a guarda <code>is_active_user()</code> que as funções irmãs já usam.
P2	Retirar o fluxo de token legado e os helpers de viagem do papel compartilhado	Achados 3 e 4	Sete funções sem chamador de produção ou sem consumidor de UI, todas concedidas a <code>authenticated</code> . Fechar o ACL das sete de uma vez elimina dois achados e reduz a superfície de RPC de forma permanente. Aplicar a mesma disciplina da migration 296.
P2	Remover o fallback para a <code>service_role</code> key e girar a chave se necessário	Achado 5	Substituir o <code>?? SUPABASE_SERVICE_ROLE_KEY</code> por falha explícita na inicialização, no padrão que <code>portalEmail.ts:47</code> já adota. Se a chave já tiver sido usada como secret header em produção, girá-la é parte da correção, não um passo opcional.
P2	Automatizar a verificação que esta auditoria fez à mão	Estrutural	A fragilidade central é que a guarda de cada função <code>SECURITY DEFINER</code> depende de disciplina humana em 201 funções. O script de replay usado aqui (<code>docs/security-audit/verificar_guardas.py</code>) resolve o estado final de policies e grants e aponta funções <code>DEFINER</code> concedidas a <code>authenticated</code> sem guarda, seguindo cadeias de delegação. Rodá-lo no CI transforma a próxima ocorrência desta classe em build vermelho, em vez de outra auditoria.
P3	Unificar a comparação de segredos das <code>Edge Functions</code>	Achado 6	Mover <code>timingSafeEqual</code> — hoje triplicado — para <code>_shared/</code> e usá-lo nas quatro funções com bearer secret. Resolve o achado e remove a duplicação.
P3	Limpar os sinks latentes e os segredos de script local	Achados 7 e 8	Remover <code>printDocumentElement</code> , que não tem chamador, e parametrizar as credenciais do shim local. Nenhum dos dois é explorável hoje; ambos são higiene que evita que uma mudança futura os torne relevantes.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar. Os oito achados foram agrupados em **cinco issues**: achados da mesma classe, com a mesma correção e o mesmo revisor, entram juntos para não gerar tickets redundantes. Cada bloco começa em `--- ISSUE n ---` e termina em `--- FIM ISSUE n ---`; copie tudo que está entre os dois delimitadores, sem incluí-los.

ISSUE 1 · [Segurança] portal_billing_gate expõe e-mail e IDs de qualquer cliente por B/L arbitrário

```
--- ISSUE 1 ---

## [Segurança] portal_billing_gate expõe e-mail e IDs de qualquer cliente por B/L arbitrário

**Labels:** `security`, `severidade:alta`

### Problema

A função é SECURITY DEFINER — portanto ignora a RLS de `bls` e de `customer_portal_accounts` — está concedida a `authenticated` e não tem nenhuma guarda no corpo: não chama `is_active_read_user()`, não chama `current_portal_customer_id()`, e não confronta o `customer_id` do B/L com o do chamador. O único parâmetro é o ID do B/L, escolhido livremente por quem chama.

O cliente do Portal recebe exatamente o mesmo role `authenticated` que o operador interno (ADR 0013 e migration 257), e o PostgREST publica toda função concedida a esse role. Um cliente autenticado no Portal chama, direto por HTTP, `POST /rest/v1/rpc/portal_billing_gate` com o ID de um B/L que não é dele e recebe de volta o `customer_id`, o `account_id` e o e-mail de recuperação do cliente dono daquele B/L. O ID do B/L é textual e sequencial no formato usado pelo próprio projeto (`CSC000000001`, `CSC000000002` em `.env.example`), o que torna a enumeração trivial: um laço sobre a faixa devolve o cadastro de contato de toda a carteira.

### Evidência

- `supabase/migrations/325_clientes_portal_disputes_alerts.sql:24-67`

```ts
CREATE OR REPLACE FUNCTION public.portal_billing_gate(p_bl_id TEXT)
RETURNS JSONB
LANGUAGE plpgsql STABLE SECURITY DEFINER
SET search_path = public, pg_temp
AS $function$
...
 SELECT customer_id INTO v_customer_id FROM public.bls WHERE id = p_bl_id;
...
RETURN jsonb_build_object(
 'allowed', v_reason IS NULL,
 'reason', v_reason,
 'customer_id', v_customer_id,
 'account_id', v_account.id,

```

```
'recovery_email', v_email -- <== e-mail real do cliente
);
END;
$function$;

REVOKE ALL ON FUNCTION public.portal_billing_gate(TEXT) FROM PUBLIC, anon;
GRANT EXECUTE ON FUNCTION public.portal_billing_gate(TEXT) TO authenticated, service_role;
```



### Impacto



Vazamento cross-tenant de PII (e-mail de contato) e do mapa cliente↔B/L de toda a operação. O e-mail de recuperação é justamente o endereço que recebe os links de recuperação de senha do Portal, o que transforma o vazamento em insumo direto para phishing dirigido contra a conta correspondente.



### Condições de exploração



Nenhuma condição especial. Basta uma conta de Portal ativa (ou qualquer sessão interna). Não depende de feature flag nem de configuração insegura.



### Correção sugerida



A função só tem consumidor interno (o gate de faturamento). Revogue o acesso do papel compartilhado e mantenha apenas `service_role`, como as migrations 257 e 296 já fizeram com `bl_has_portal_release` e `check_provision_rate_limit`:



```
`REVOKE ALL ON FUNCTION public.portal_billing_gate(TEXT) FROM PUBLIC, anon, authenticated;`
```



Se algum chamador interno precisar dela com sessão de usuário, adicione no início do corpo a guarda padrão do projeto: `IF NOT public.is_active_read_user() THEN RAISE EXCEPTION 'Sem permissão.' USING ERRCODE = '42501'; END IF;`



### Critérios de aceite



- [ ] portal_billing_gate não aparece mais com EXECUTE para authenticated em pg_proc/ACL.
- [ ] Uma sessão de cliente do Portal recebe 42501 (ou 404 do PostgREST) ao chamar a RPC.
- [ ] O fluxo interno de emissão de fatura que usa o gate continua passando nos testes.
- [ ] Um teste de regressão cobre a chamada negada a partir de sessão do Portal.
- [ ] `npm run lint`, `npm test` e `npm run build` passam.
- [ ] A migration de correção traz cabeçalho com intenção, afetados e rollback (convenção do projeto).

```

--- FIM ISSUE 1 ---

ISSUE 2 · [Segurança] refresh_voyage_status_from_terminal_scales altera status de viagem sem verificar privilégio

--- ISSUE 2 ---

[Segurança] refresh_voyage_status_from_terminal_scales altera status de viagem sem verificar privilégio

****Labels:**** `security`, `severidade:media`

Problema

É uma função de ****escrita****, SECURITY DEFINER (portanto acima da RLS de `voyages`), concedida a `authenticated`, que recebe o `voyage\_id` como único parâmetro e não verifica nem quem chama nem se o chamador tem relação com aquela viagem. Ela é o reconciliador chamado por triggers e pelo detector agendado – mas o grant a `authenticated` a torna também um endpoint público do PostgREST.

Qualquer sessão autenticada, inclusive a de um cliente do Portal, pode chamá-la em laço sobre IDs de viagem (inteiros sequenciais) e forçar a transição de `status` para `completed` ou `active`. Vale notar que a escrita não é arbitrária: o valor gravado é derivado do estado real das escalas, então o atacante não escolhe o status. O que ele obtém é a capacidade de ****disparar a transição fora de hora**** e de resolver, pelo mesmo caminho, os itens de alerta associados – em cima de qualquer viagem da operação.

Evidência

- `supabase/migrations/342\_atracacao\_alertas.sql:7-47`

``ts

```
CREATE OR REPLACE FUNCTION public.refresh_voyage_status_from_terminal_scales(p_voyage_id BIGINT)
RETURNS VOID
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, pg_temp
AS $function$
...
BEGIN
  -- nenhuma verificação de identidade ou de papel
  ...
  IF v_active_scales > 0 THEN
    UPDATE public.voyages
    SET status = CASE WHEN v_pending_scales = 0 THEN 'completed' ELSE 'active' END
    WHERE id = p_voyage_id AND status <> 'cancelled';
```

```
END IF;
END;
$function$;

REVOKE ALL ON FUNCTION public.refresh_voyage_status_from_terminal_scales(BIGINT) FROM PUBLIC, anon;
GRANT EXECUTE ON FUNCTION public.refresh_voyage_status_from_terminal_scales(BIGINT) TO
    authenticated, service_role;
` ``

### Impacto

Integridade do estado operacional e da fila de alertas. Uma viagem marcada como `completed` antes
do tempo, ou um alerta operacional resolvido por um terceiro, corrompe o painel que a operação
usa para decidir. Como a escrita não passa pelo caminho normal, ela também não deixa o rastro de
autoria que os fluxos internos deixam.

### Condições de exploração

Nenhuma condição especial; qualquer sessão autenticada. O efeito depende do estado das escalas da
viagem alvo, então nem toda chamada produz mudança.

### Correção sugerida

Manter o grant apenas para `service_role` (o detector agendado) ou, se houver chamador interno com
sessão de usuário, adicionar a guarda que as funções irmãs da mesma família já usam –
`resolve_alert_item` valida e levanta 42501:
`IF auth.uid() IS NULL OR NOT public.is_active_user() THEN RAISE EXCEPTION 'Sem permissão.' USING
    ERRCODE = '42501'; END IF;`

### Critérios de aceite

- [ ] A RPC recusa chamada de sessão do Portal e de usuário interno inativo.
- [ ] O detector agendado (pg_cron/service_role) continua reconciliando o status normalmente.
- [ ] Os triggers que chamam a função internamente continuam funcionando.
- [ ] Teste de regressão cobre a chamada negada.
- [ ] `npm run lint`, `npm test` e `npm run build` passam.
- [ ] A migration de correção traz cabeçalho com intenção, afetados e rollback (convenção do
    projeto).

--- FIM ISSUE 2 ---
```

ISSUE 3 · [Segurança] Fechar o ACL das RPCs SECURITY DEFINER sem guarda expostas a authenticated

--- ISSUE 3 ---

[Segurança] Fechar o ACL das RPCs SECURITY DEFINER sem guarda expostas a authenticated

****Labels:**** `security`, `severidade:media`

> ****Por que agrupada:**** Agrupa os achados 3 e 4: mesma classe (função SECURITY DEFINER concedida a `authenticated` sem guarda no corpo) e mesma correção (fechar o ACL). Resolver os dois no mesmo commit evita duas revisões de migration para a mesma decisão.

Problema

****[MÉDIA]** Leitura de programação de viagem sem guarda contorna o portão show_on_portal

As duas funções são SECURITY DEFINER sem guarda, concedidas a `authenticated`, e leem `audit\_logs` – cuja RLS restringe a leitura a usuário interno ativo. Por serem DEFINER, essa RLS não se aplica.

O ponto que torna isso uma falha de isolamento, e não apenas uma inconsistência, é o portão `voyages.show\_on\_portal` (migration 172), criado exatamente para decidir qual programação o cliente pode ver, e reforçado pela migration 257, que removeu as policies `USING (true)` de `vessel\_schedules` por contornarem esse portão. Estas duas funções contornam o mesmo portão pelo outro lado: um cliente do Portal itera `voyage\_id` (inteiro sequencial) e obtém os portos de escala elegíveis e a data de ETA de ****qualquer**** viagem, inclusive das que a operação decidiu não publicar.

****[MÉDIA]** portal_check_auth_method: oráculo de enumeração de clientes e vazamento de portal_email (código morto ainda publicado)

A própria migration 053 nasceu para fechar a enumeração aqui: o cabeçalho diz "remove user enumeration (...) impedindo varredura de CNPJs de clientes via API anon". A correção unificou a resposta de CNPJ inexistente e conta inativa, e a migration 093 depois revogou `anon` de todas as funções SECURITY DEFINER. ****O grant a `authenticated` nunca foi revogado****, e a unificação não cobre o terceiro desfecho: conta ****ativa**** devolve `method: supabase\_auth` mais o `portal\_email`.

O CNPJ é dado público (consultável na Receita Federal), então o parâmetro não é segredo. Qualquer sessão autenticada – inclusive um cliente do Portal – varre uma lista de CNPJs e aprende, para cada um, se a empresa tem conta ativa no Portal e qual o endereço registrado. A função é além disso ****código morto****: a própria migration 093 registra que ela é "do fluxo de token legado e não tem mais nenhum call site no frontend", o que a auditoria confirmou – o frontend usa `portal\_get\_session\_overview\_v2`.

Evidência

```
- `supabase/migrations/326_voyage_operation_alerts.sql:15-49`
- `supabase/migrations/326_voyage_operation_alerts.sql:53-103`

```ts
-- 1. Helper: PODs elegíveis da viagem
CREATE OR REPLACE FUNCTION public.get_voyage_eligible_pods(p_voyage_id BIGINT)
RETURNS TABLE(pod TEXT)
LANGUAGE sql STABLE SECURITY DEFINER
SET search_path = public, pg_temp
AS $$
 WITH latest_states AS (
 SELECT DISTINCT ON (entity_id, field_name) entity_id, field_name, new_value
 FROM public.audit_logs -- <== RLS ignorada, sem guarda
 WHERE entity_type = 'voyage_pod_schedule'
 AND entity_id LIKE p_voyage_id || ':%'
)
 ...
$$;
REVOKE ALL ON FUNCTION public.get_voyage_eligible_pods(BIGINT) FROM PUBLIC, anon;
GRANT EXECUTE ON FUNCTION public.get_voyage_eligible_pods(BIGINT) TO authenticated, service_role;

-- 2. Helper: menor ETA brasileiro
CREATE OR REPLACE FUNCTION public.get_voyage_first_brazilian_eta(p_voyage_id BIGINT)
RETURNS DATE ... SECURITY DEFINER ... -- idem: sem guarda
GRANT EXECUTE ON FUNCTION public.get_voyage_first_brazilian_eta(BIGINT) TO authenticated,
 service_role;
```

- `supabase/migrations/053_security_hardening.sql:16-49`

```ts
CREATE OR REPLACE FUNCTION public.portal_check_auth_method(p_cnpj_cpf TEXT)
RETURNS JSONB LANGUAGE plpgsql SECURITY DEFINER
SET search_path = public, pg_temp
AS $$
BEGIN
 SELECT a.auth_user_id, a.portal_email, a.active INTO v_account
 FROM public.customer_portal_accounts AS a
```

```

JOIN public.customers AS c ON c.id = a.customer_id
WHERE c.cnpj_cpf = p.cnpj_cpf; -- <= CNPJ arbitrário, dado público

IF NOT FOUND OR NOT v_account.active THEN
 RETURN jsonb_build_object('method', 'none');
END IF;
IF v_account.auth_user_id IS NOT NULL THEN
 RETURN jsonb_build_object('method', 'supabase_auth',
 'portal_email', v_account.portal_email);
END IF;
RETURN jsonb_build_object('method', 'legacy_token');
END;
$$;

REVOKE ALL ON FUNCTION public.portal_check_auth_method(TEXT) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.portal_check_auth_method(TEXT) TO anon, authenticated;
```

```

Impacto

Vazamento de programação comercial (rota e datas) que o portão de publicação existe para controlar. Para um cliente que é concorrente de outro embarcador na mesma rota, é informação de valor competitivo.

Enumeração da carteira de clientes com Portal ativo e coleta dos endereços associados. É o mesmo risco que a migration 053 tentou eliminar, sobrevivendo por um grant que ela não tocou. Superfície de ataque mantida por código sem consumidor.

Condições de exploração

Qualquer sessão autenticada. O retorno é limitado a códigos de porto e datas – não expõe B/L, cliente nem valores.

Qualquer sessão autenticada. A migration 093 já fechou o caminho não autenticado (`anon`), então o pré-requisito é ter uma conta – o que um cliente do Portal tem por definição.

Correção sugerida

São helpers de detector: o consumidor natural é `service\_role`. Revogue `authenticated` das duas. Se a UI interna precisar delas, adicione `WHERE public.is\_active\_read\_user()` no corpo SQL – o

mesmo padrão que a migration 257 aplicou em `list\_billing\_runs`.

Revogar e, de preferência, remover. O fluxo de token legado inteiro (`portal\_check\_auth\_method`, `portal\_get\_session\_overview`, `portal\_list\_pending\_bls`, `portal\_logout`, `resolve\_customer\_portal\_session`) está sem chamador; a migration 296 já aplicou essa disciplina às funções `\*\_legacy` e estas ficaram de fora só porque o nome não casa com o padrão.

Critérios de aceite

- [] As duas funções não têm mais EXECUTE para authenticated, ou passaram a exigir is_active_read_user().
- [] Sessão de cliente do Portal não consegue mais obter POD/ETA de viagem não publicada.
- [] run_alert_detectors() e os reconciliadores continuam funcionando.
- [] portal_check_auth_method não tem mais EXECUTE para authenticated.
- [] As demais RPCs do fluxo de token legado tiveram o ACL fechado no mesmo commit.
- [] Nenhum fluxo do Portal (login, recuperação, ativação) regride nos testes.
- [] docs/RASTREABILIDADE.md registra a retirada.
- [] `npm run lint`, `npm test` e `npm run build` passam.
- [] A migration de correção traz cabeçalho com intenção, afetados e rollback (convenção do projeto).

--- FIM ISSUE 3 ---

ISSUE 4 · [Segurança] Endurecer a autenticação por bearer secret das Edge Functions

--- ISSUE 4 ---

[Segurança] Endurecer a autenticação por bearer secret das Edge Functions

****Labels:**** `security`, `severidade:media`

> ****Por que agrupada:**** Agrupa os achados 5 e 6: os dois estão na autenticação por bearer secret das Edge Functions e são resolvidos no mesmo arquivo compartilhado.

Problema

****[MÉDIA]** Segredo de webhook e de cron caem para a chave service_role quando não configurados**

Não é um segredo escrito no código, e sim o padrão que o pedido da auditoria chama de `_default` que vira segredo real se não for sobrescrito – com o agravante de que o default aqui é a credencial ****mais privilegiada do projeto****. Se `NOTIFY_WEBHOOK_SECRET` ou `RECALC_CRON_SECRET` não estiver definida, a função passa a aceitar como bearer a `SUPABASE_SERVICE_ROLE_KEY`. Para o webhook funcionar nesse estado, um operador tem de ****colar a service_role key no campo de secret header**** do Database Webhook e do agendador – e a partir daí ela viaja como bearer em cada disparo, fica gravada na configuração do painel e tende a aparecer em log de requisição.

Uma chave que só deveria existir dentro do runtime da função passa a circular como token de transporte. Não há nenhuma validação de inicialização que recuse essa configuração: o código não distingue "segredo dedicado ausente" de "segredo dedicado configurado", então a degradação é silenciosa. Note que isto ****não**** é um bypass de autenticação – quem não conhece nenhuma das duas chaves continua recebendo 401.

****[BAIXA]** portal-daily-digest compara o segredo com `!=` em vez de comparação em tempo constante**

É a única das quatro funções protegidas por bearer secret que compara com `!=`. As outras três – `alerts-detector:3-11`, `notify-invoice-issued:58-66` e `recalc-demurrage-ptax:26-34` – implementam `timingSafeEqual`, e a de `notify-invoice-issued` até documenta o motivo em comentário. A comparação de strings do JavaScript sai no primeiro byte diferente, o que em tese permite recuperar o segredo byte a byte pela variação do tempo de resposta.

Na prática a exploração é pouco realista: o ruído de rede e o cold start do Edge Runtime dominam a diferença de nanossegundos. O achado é registrado como ****inconsistência de padrão****: a defesa foi adotada como convenção no projeto e este ponto ficou de fora.

Evidência

```

- `supabase/functions/notify-invoice-issued/index.ts:77-78`
- `supabase/functions/recalc-demurrage-ptax/index.ts:61-62`

```ts
// supabase/functions/notify-invoice-issued/index.ts:77-78
const webhookSecret =
 Deno.env.get('NOTIFY_WEBHOOK_SECRET') ?? Deno.env.get('SUPABASE_SERVICE_ROLE_KEY') ?? ''

// supabase/functions/recalc-demurrage-ptax/index.ts:61-62
const serviceRoleKey = Deno.env.get('SUPABASE_SERVICE_ROLE_KEY') ?? ''
const cronSecret = Deno.env.get('RECALC_CRON_SECRET') ?? serviceRoleKey
```

- `supabase/functions/portal-daily-digest/index.ts:7-9`

```ts
const expectedSecret = Deno.env.get('PORTAL_DIGEST_SECRET')
const providedSecret = req.headers.get('Authorization')?.replace(/^Bearer\s+/i, '')
if (!expectedSecret || providedSecret !== expectedSecret) return new Response(JSON.stringify({
 error: 'Unauthorized' }), { status: 401 })
```

### Impacto

Ampliação da superfície de exposição da credencial que ignora toda a RLS do projeto. O comprometimento dela é total: leitura e escrita irrestritas em todas as 92 tabelas.

Baixo. Recuperação teórica do PORTAL_DIGEST_SECRET, que dispara o envio do resumo diário aos usuários internos.

### Condições de exploração

Requer a configuração insegura (variável dedicada ausente) para se materializar. Não é explorável remotamente por si só; é uma falha de higiene de credencial que amplia o raio de outro incidente.

Exige medição de tempo através da rede contra ruído alto. Não demonstrada.

### Correção sugerida

Remover o fallback e falhar de forma observável, como `_shared/portalEmail.ts:47` já faz com `PORTAL_FROM_EMAIL`:



```
const webhookSecret = Deno.env.get('NOTIFY_WEBHOOK_SECRET')`
if (!webhookSecret) return json(500, { error: 'Webhook não configurado.' })`
```



Depois, girar a service_role key se ela já tiver sido usada como secret header em produção.

Extrair o `timingSafeEqual` já duplicado em três funções para `_shared/` e usá-lo nas quatro – resolve o achado e remove a triplicação no mesmo movimento.

### Critérios de aceite



- [ ] Nenhuma Edge Function usa SUPABASE_SERVICE_ROLE_KEY como fallback de segredo de webhook ou cron.
- [ ] Ausência da variável dedicada resulta em 500 explícito, não em aceitação silenciosa da service_role.
- [ ] A service_role key foi girada caso tenha sido configurada como secret header.
- [ ] docs/ARCHITECTURE.md lista as variáveis obrigatórias por função.
- [ ] As quatro Edge Functions com bearer secret usam a mesma comparação em tempo constante.
- [ ] A implementação vive em supabase/functions/_shared/ e não está mais duplicada.
- [ ] `npm run lint`, `npm test` e `npm run build` passam.
- [ ] A migration de correção traz cabeçalho com intenção, afetados e rollback (convenção do projeto).



--- FIM ISSUE 4 ---

```

ISSUE 5 · [Segurança] Higiene: segredos de script local e sink latente de HTML na impressão

--- ISSUE 5 ---

[Segurança] Higiene: segredos de script local e sink latente de HTML na impressão

****Labels:**** `security`, `severidade:baixa`

> ****Por que agrupada:**** Agrupa os achados 7 e 8: nenhum é explorável hoje. Uma issue só evita dois tickets de baixa prioridade para trabalho de limpeza.

Problema

****[BAIXA]** Segredos fixos em scripts do ambiente local de auditoria de design

São credenciais fixas de um emulador local do Supabase usado pela skill de auditoria de design. O próprio arquivo se declara "Not for production use" na segunda linha, o shim escuta em `127.0.0.1:54321` e nada em `vercel.json`, em `firebase.json` ou no workflow de CI o executa ou o publica.

0 registro é higiene, não vulnerabilidade explorável: um segredo de assinatura de JWT literal no repositório é exatamente o padrão que uma varredura automatizada de segredos aponta, e convém que o motivo de ele ser inofensivo esteja escrito em vez de precisar ser redescoberto a cada auditoria.

****[INFORMATIVA]** innerHTML reinjetado em document.write no fluxo de impressão (sink latente, não explorável hoje)

Registrado como informativo porque ****não é explorável no estado atual****, e a auditoria confirmou as três condições que sustentam isso: (a) não existe nenhum `dangerouslySetInnerHTML` no repositório, então o `innerHTML` lido é DOM que o React renderizou e já escapou; (b) o `title` interpolado sem escape em `printDocument.ts:8` não chega a ser um vetor porque a função ****não tem nenhum chamador**** — é código morto; (c) em `Demurrage.tsx:157`, que é o caminho vivo, o título é uma string literal.

0 que se registra é a fragilidade da construção. A janela aberta por `window.open('')` é ****same-origin****: se um dia alguém introduzir um `dangerouslySetInnerHTML` na árvore do documento de impressão, ou passar um título vindo do usuário para a função morta, o XSS resultante roda no mesmo contexto da aplicação, com acesso ao token da sessão no storage. O projeto não tem biblioteca de sanitização instalada, então não existe rede de segurança para esse caso.

Evidência

```
- `scripts/design-audit/sb-shim.cjs:18`
- `scripts/design-audit/win/local-stack.ps1:32`
- `scripts/design-audit/win/local-stack.ps1:128`

```ts
// scripts/design-audit/sb-shim.cjs:1-2 e :18
// Minimal local Supabase emulator (PostgREST subset + GoTrue password auth)
// for UI auditing against a local Postgres. Not for production use.
const JWT_SECRET = 'local-audit-jwt-secret-local-audit-jwt-secret-32'

scripts/design-audit/win/local-stack.ps1:32
$env:PGPASSWORD = "postgres"
```

- `src/lib/printDocument.ts:1-11`
- `src/pages/Demurrage.tsx:149-162`

```ts
// src/lib/printDocument.ts:1-8
export function printDocumentElement(element: HTMLElement, title: string) {
 const printWindow = window.open('', '_blank', 'width=900,height=1100')
 ...
 printWindow.document.write(`<!doctype html><html><head><title>${title}</title>
 ...<div class="invoice-print-content">${element.innerHTML}</div></body></html>`)

// src/pages/Demurrage.tsx:157 – mesma construção, título literal
printWindow.document.write(`...<title>Fatura de Demurrage</title>...
 <div class="invoice-print-content">${content.innerHTML}</div>...`)
```

### Impacto

Nenhum em produção. O risco real é de contágio: alguém reaproveitar o shim ou o script fora do contexto local.

Nenhum hoje. O valor do registro é impedir que a construção se torne o vetor de uma mudança futura aparentemente inócua.

### Condições de exploração

Não explorável remotamente. Exige que o shim seja executado e exposto fora de localhost – o que nenhum caminho do projeto faz.

Não explorável no código atual. Depende da introdução de um sink de HTML bruto na árvore impressa ou de um chamador que passe título controlado pelo usuário.

### Correção sugerida

Ler as duas credenciais de variável de ambiente com default apenas quando `NODE_ENV !== 'production'`, e recusar a inicialização fora de localhost. Alternativamente, manter como está e registrar a exceção no arquivo de configuração da varredura de segredos, para que o achado não volte a cada execução.

Remover `printDocumentElement`, que não tem chamador, ou escapar o `title` antes de interpolar. A CSP de `vercel.json` (`script-src 'self'`) já bloqueia script inline e é a mitigação que hoje segura o caso – vale registrar essa dependência no comentário da função.

### Critérios de aceite

- [ ] O shim recusa iniciar se o host de bind não for loopback.
- [ ] JWT_SECRET e PGPASSWORD vêm do ambiente, com default só fora de produção.
- [ ] Ou: a exceção está registrada na configuração da varredura de segredos, com justificativa.
- [ ] printDocumentElement foi removida, ou o title é escapado antes da interpolação.
- [ ] Existe um comentário registrando que a impressão depende da CSP como mitigação.
- [ ] `npm run lint`, `npm test` e `npm run build` passam.
- [ ] A migration de correção traz cabeçalho com intenção, afetados e rollback (convenção do projeto).

--- FIM ISSUE 5 ---
```